

reflector_design

Multicast Discovery Reflector — Remote-Side Design (VPN-LAN Phase 5)

Revision: 1 **Last modified:** 2026-07-01T00:00:00Z **Status:** Design — reflector NOT deployed (operator-gated §11.4.122); local discovery test honest-SKIPs until a reflector is configured (§11.4.3) **Authority:** Inherits constitution/Constitution.md per §11.4.35. Phase 5 of PLAN.md §5 — the remote-side multicast discovery reflector that lets helix_proxy-side clients enumerate services living on the sword VPN-internal subnet. **Feature workstream:** feature/vpn-aware-dynamic-routing (§11.4.167) **Companion:** plan PLAN.md §2 routing map (the “Multicast discovery → REFLECT” row) + §5 Phase 5 · the discovered devices are then driven by Phase 6 (Google Cast / DIAL) · the structural-impossibility sibling is miracast_verdict.md · local proof: ../.../.../tests/vpn_lan/discovery_reflect.sh

1. Problem statement (FACT, §11.4.6)

helix_proxy’s VPN is **L3-routed** (WireGuard + L2TP/PPP over ppp0; reachable subnet 10.0.0.0/8; sword host 10.6.100.221) — see PLAN.md §2. Unicast services (SMB, NFS, FTP, SFTP, IMAP, SMTP, POP3, WebDAV, Cast control, DIAL control, ADB) route across it as ordinary IP packets. **Service discovery, however, does not** — the four mainstream discovery protocols in scope are all **multicast**, and a router (which is exactly what an L3 VPN is) **does not forward these multicast groups across a subnet boundary by default**. So a helix_proxy-side client that runs an avahi-browse / SSDP M-SEARCH sees only its own local subnet, never the remote VPN-internal subnet where the target devices actually live.

The discovery protocols and their multicast well-known endpoints (FACT — stable, standardized values):

Protocol	Transport	IPv4 group	Port	Governing spec
mDNS (Bonjour name resolution)	UDP	224.0.0.251	5353	RFC 6762
DNS-SD (service enumeration, layered on mDNS)	UDP	224.0.0.251	5353	RFC 6763
SSDP / UPnP (device discovery)	UDP	239.255.255.250	1900	UPnP Device Architecture
WS-Discovery (SOAP-over-UDP discovery)	UDP	239.255.255.250	3702	OASIS WS-Discovery

2. WHY multicast does not cross an L3 VPN (the routing argument, FACT)

Two independent standard-mandated properties make these groups **subnet-local by design** — a router (the VPN) is *required* to drop them, not merely configured not to forward them:

1. **mDNS / DNS-SD use a link-local multicast group.** 224.0.0.251 sits inside 224.0.0.0/24, the IANA-registered **Local Network Control Block** (RFC 5771). Traffic to this range is **link-local**: it is sent with **IP TTL = 1** (RFC 6762 §11 requires mDNS queries/responses to use TTL 255 for its own integrity check but the group itself is link-scoped, and in practice the packets never leave the link), and routers **MUST NOT forward** Local-Network-Control-Block multicast off the originating link. A packet with TTL 1 is decremented to 0 at the first router hop and discarded before it can reach the peer. **There is nothing for the L3 VPN to route** — the datagram is dead at the first hop by the standard's own rules.
2. **SSDP / WS-Discovery use an administratively-scoped group with a tiny hop budget.** 239.255.255.250 is in 239.0.0.0/8, the **Administratively Scoped** / organization-local multicast range (RFC 2365 / RFC 5771). The UPnP Device Architecture specifies SSDP messages be sent with a **small default TTL (2)** so they stay within the local site, and —

decisively — **no multicast routing exists across the VPN tunnel to carry 239.0.0.0/8 between subnets** in the first place. Even were the TTL larger, an ordinary unicast-only L3 VPN carries no IGMP/PIM multicast-routing state, so the group is never propagated to the remote subnet's clients.

Consequence (the recon finding, [PLAN.md §2 / §11](#)): “Multicast discovery — *REFLECT* on the remote (device-side) network — routers do not forward it across L3.” Unicast routes; multicast must be **reflected** onto the subnet where the querying client lives. This is not a helix_proxy limitation — it is a property of IP multicast scoping shared by every L3-routed network.

3. Reflector architecture (the fix)

The fix is to place a **multicast discovery reflector on the REMOTE (device-side) subnet** — the 10.0.0.0/8 side where the discoverable services physically live — that re-emits the discovery traffic so a helix_proxy-side client, reachable over the routed VPN, can enumerate the remote services. The reflector is **two cooperating components**, one per multicast family, because mDNS (link-local 224.0.0.251/5353) and SSDP/WS-Discovery (admin-scoped 239.255.255.250/1900,3702) are different groups needing different relays:

3.1 mDNS / DNS-SD — Avahi reflector (enable-reflector=yes) — FACT

Avahi (the standard Linux mDNS/DNS-SD stack) ships a first-class **reflector mode**. In `avahi-daemon.conf`:

```
[reflector]
enable-reflector=yes
```

With `enable-reflector=yes` the daemon **repeats mDNS packets it sees on one interface out of every other interface it manages**, bridging the 224.0.0.251:5353 group across the interface boundary. Deployed on a host that has a foot on the remote device subnet **and** a routed/bridged path the helix_proxy client can reach, it reflects the remote devices' Bonjour/DNS-SD advertisements so `avahi-browse` on the helix_proxy side resolves them. This is a **documented, supported Avahi feature** — not a bespoke hack.

- **Scope discipline (FACT):** the reflector repeats *all* mDNS it sees, so it **MUST** be constrained to only the interfaces that actually need bridging (`allow-interfaces` / `deny-interfaces` in `avahi-daemon.conf`) to avoid flooding unrelated links. Over-broad

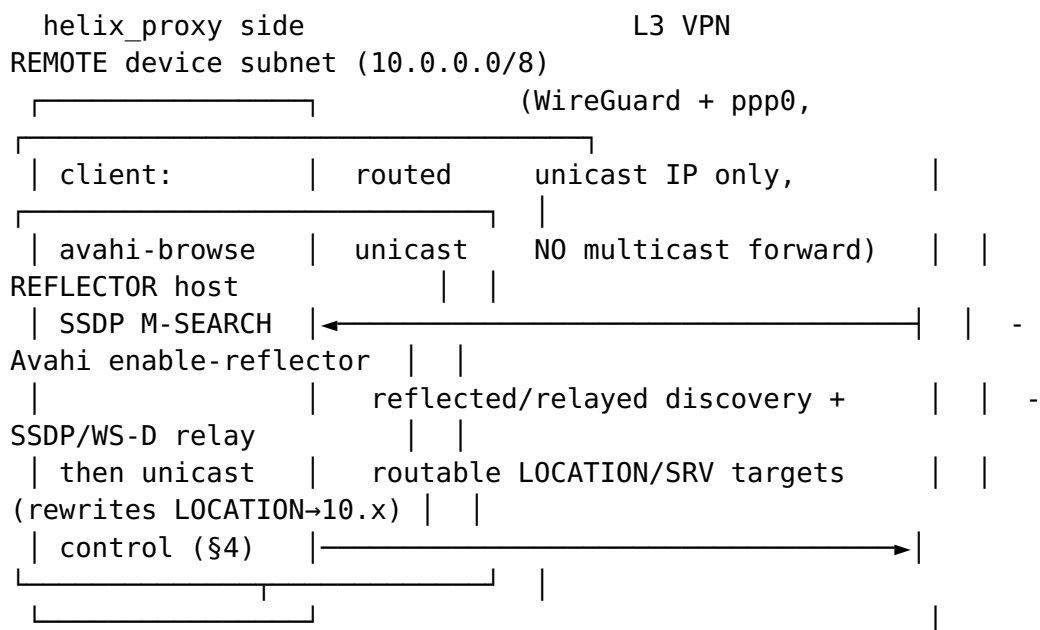
reflection is a noise/security concern, so the interface allow-list is part of the deployed config, not an afterthought.

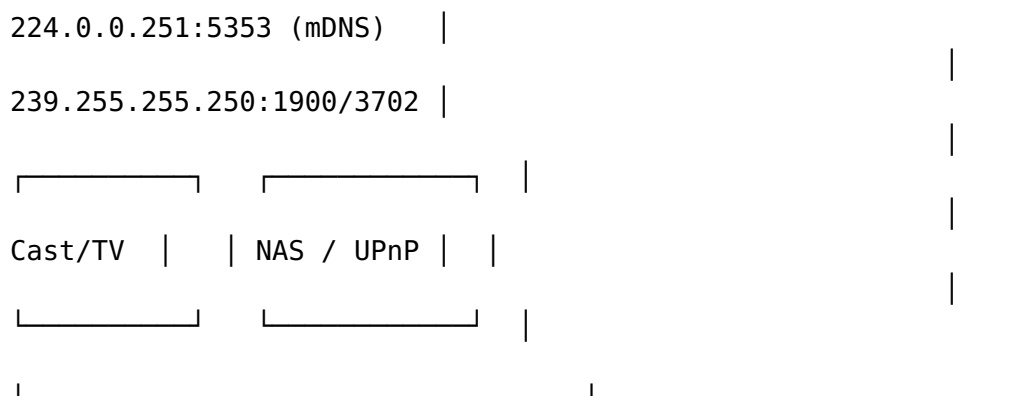
3.2 SSDP / UPnP (1900) + WS-Discovery (3702) — an SSDP relay — INFERENCE on exact tool

Avahi handles **only** mDNS. SSDP (239.255.255.250:1900) and WS-Discovery (239.255.255.250:3702) need a **separate multicast relay** on the same remote subnet. Two mechanisms, in order of preference:

- **A dedicated SSDP/UPnP relay daemon** that listens for M-SEARCH on one interface and re-issues advertisements/responses onto the client-facing interface — the cleanest fit because it understands SSDP's M-SEARCH / NOTIFY semantics and can rewrite the LOCATION header so the advertised URL is a **routable 10.x address** the helix_proxy client can then reach over unicast. (**INFERENCE — §11.4.6:** the specific relay daemon is a deployment-time choice; the *requirement* — an SSDP-aware relay that rewrites LOCATION to a routable address — is FACT, the exact binary is not yet pinned and will be selected + cited when the reflector is actually deployed per §11.4.150.)
- **A static multicast router (smcroute)** as the generic fallback: it statically forwards a multicast group between two interfaces without SSDP awareness. This carries the 239.255.255.250 datagrams across the interface boundary but does **not** rewrite LOCATION URLs, so it only helps when the advertised URLs already carry routable addresses. (**INFERENCE** on suitability for a given site.)

3.3 Deployment topology





The reflector lives **on the remote subnet** (where physics allows it to see 224.0.0.251 / 239.255.255.250 locally) and re-emits toward the helix_proxy-reachable side. helix_proxy itself deploys and runs nothing on the remote host autonomously (§3.4).

3.4 Containerization (containers submodule §11.4.76, rootless §11.4.161)

Per PLAN.md §5 T5.1 + Phase 10, the reflector is packaged and booted **only** via the submodules/containers orchestration layer — **never** an ad-hoc podman/docker invocation:

- The Avahi-reflector + SSDP-relay images boot **on-demand** through the submodule’s pkg/boot / pkg/compose / pkg/health primitives (the on-demand-infra invariant, §11.4.76) — the test/deploy entry point brings the reflector up, the operator is never asked to run podman by hand.
- The runtime is **rootless Podman** (§11.4.161) — no sudo, no rootful Docker.
- Any missing capability (e.g. the SSDP relay is not yet an image the submodule can model) is added by **extending submodules/containers upstream** (§11.4.74 extend-don’t-reimplement), never worked around with a raw command in this repo.
- Config (the remote subnet, the allowed interfaces, the LOCATION-rewrite target) is **injected** into the container (env / config file, §11.4.28 decoupling) — the submodule stays project-agnostic and the reflector image carries **no** helix_proxy-specific hardcoding.

3.5 Operator-gated deployment (§11.4.122)

Deploying a reflector on a remote host CHANGES that host (it starts a daemon, joins multicast groups, re-emits traffic onto its subnet). Under §11.4.122 (no silent change to any remote/connected host) **and** PLAN.md §1 hard-constraint 2, helix_proxy **MUST NOT** deploy the reflector autonomously. The sequence is:

1. **Ask first (§11.4.66 interactive options), BEFORE any remote deployment** — present the operator a keep/deploy

decision with the concrete blast radius (which host, which subnet, which interfaces, that a daemon will be started and multicast re-emitted), and the teardown path.

2. **Only on explicit operator approval** does the containerized reflector boot on the operator-designated remote host via submodules/containers.
3. Until then the reflector is **not deployed**, and the local discovery test honest-SKIPs (`feature_disabled_by_config` when no reflector is configured — §11.4.3 / §11.4.69), never a fake PASS.

This mirrors the whole feature's autonomous-vs-operator-gated split (PLAN.md §8): the **design + the honest-SKIP-gated local test are autonomous now**; the **remote deployment is parked as operator-gated** (§11.4.21) and surfaced via §11.4.66 when reached.

4. Honest boundary — discovery is not control (§11.4.6)

The reflector solves **discovery only**. Its job ends the moment a remote service is *enumerated* on the `helix_proxy`-side client. Everything after that is **routable unicast** and does **not** need the reflector:

- **Google Cast** — discovery is `_googlecast._tcp` mDNS (reflected here); once the device + its `10.x` address are known, control is **unicast TCP**: GET `http://<ip>:8008/setup/eureka_info` and CASTV2 on 8009 (TLS). These route over the L3 VPN directly (PLAN.md §5 Phase 6, driven by `chromecast_dial.sh`).
- **DIAL** — discovery is SSDP M-SEARCH (reflected here); control is plain **unicast HTTP** to the advertised LOCATION.
- **UPnP / WS-Discovery devices** (NAS, printers, media servers) — discovery is multicast (reflected here); the subsequent SOAP/HTTP control-plane and data-plane calls are **unicast** to the discovered LOCATION/endpoint.

So the reflector's **single deliverable** is: *"a remote service, and its routable address/URL, becomes visible to a helix_proxy-side discovery client."* The LOCATION/SRV target it surfaces **MUST** be a **routable 10.x address** (hence §3.2's rewrite requirement) so the follow-up unicast probe reaches the real device over the VPN. What the reflector does **not** and **cannot** do: carry the control or media planes (those are unicast and route on their own), and — per `miracast_verdict.md` — it does **nothing** for Miracast (Wi-Fi-Direct/L2, structurally-impossible over an L3 VPN, §11.4.112; the routable casting alternative is Google Cast / DIAL above).

What this design guarantees vs. does not (§11.4.6): it guarantees a **correct, standards-grounded discovery-bridging architecture** and a **local test that honestly SKIPS until a reflector exists** — it does **NOT** guarantee live enumeration until the operator approves and a reflector is actually deployed on the remote subnet (§3.5). No phase is “done” until its runtime signature verifies with captured evidence (§11.4.108) and it crosses independent review (§11.4.142).

5. Local verification (the autonomous slice — discovery_reflect.sh)

The Phase-5 test proves the **client-side enumeration path** with the exact anti-bluff discipline of every VPN-LAN test:

- **Bridge gate first (§11.4.3 / §11.4.69):** sources tests/lib/sword_bridge.sh, calls bridge_require; when the sword bridge is **down** (the default autonomous state, no .env) it prints an honest SKIP and exits 0 — **the path that runs now**. No reflector is contacted at all.
 - **Bridge up + reflector configured** (HELIX_VPN_REFLECTOR present/reachable): runs avahi-browse -rpt _services._dns-sd._udp (or a configured _googlecast._tcp) and **asserts a REAL remote service is enumerated through the reflector** — the PASS evidence is the non-empty, resolved (^=) browse output; an SSDP M-SEARCH to 239.255.255.250:1900 is captured as **supplementary** discovery context (non-scored).
 - **No reflector configured** ⇒ SKIP:feature_disabled_by_config (the reflector is operator-gated and not yet deployed — §3.5).
 - **avahi-browse absent** ⇒ SKIP:topology_unsupported (client tool missing).
 - **Never a fake PASS** — enumeration of a real service is required for PASS; an absent reflector/tool SKIPS, it never PASSES (§11.4.6 / §11.4.69).
-

Sources verified 2026-07-01

- **RFC 6762 — Multicast DNS** (mDNS; UDP 5353, group 224.0.0.251, link-local scope): <https://www.rfc-editor.org/rfc/rfc6762>
- **RFC 6763 — DNS-Based Service Discovery** (DNS-SD layered on mDNS): <https://www.rfc-editor.org/rfc/rfc6763>
- **RFC 5771 — IANA Guidelines for IPv4 Multicast Address Assignments** (224.0.0.0/24 Local Network Control Block; 239.0.0.0/8 Administratively Scoped): <https://www.rfc-editor.org/rfc/rfc5771>

- **RFC 2365 — Administratively Scoped IP Multicast** (239.0.0.0/8 organization-local scope): <https://www.rfc-editor.org/rfc/rfc2365>
- **Avahi avahi-daemon.conf — [reflector] enable-reflector=yes** (the documented mDNS reflector mode): <https://linux.die.net/man/5/avahi-daemon.conf> · project: <https://www.avahi.org/>
- **UPnP Device Architecture** (SSDP over 239.255.255.250:1900, small default TTL): <https://openconnectivity.org/developer/specifications/upnp-resources/upnp/>
- **OASIS Web Services Dynamic Discovery (WS-Discovery)** (SOAP-over-UDP, 239.255.255.250:3702): <https://docs.oasis-open.org/ws-dd/discovery/1.1/os/wsdd-discovery-1.1-spec-os.html>
- **smcroute — static multicast routing daemon** (the generic fallback relay): <https://github.com/troglobit/smcroute>

*Access date for all sources: 2026-07-01. FACT items (multicast groups, ports, RFC-mandated link-local / administratively-scoped behaviour, Avahi's enable-reflector feature) are grounded in the cited standards; items explicitly marked **INFERENCE** (§11.4.6) — the specific SSDP-relay binary selection and its suitability per site — are deployment-time choices to be pinned + cited when the reflector is actually deployed per §11.4.150, never asserted as settled here. Deep-research (§11.4.150), multi-angle: routing-layer (RFC 5771/2365 scoping), protocol-layer (RFC 6762/6763, UPnP, WS-Discovery), tooling-layer (Avahi reflector, smcroute) — access date 2026-07-01.*